

Coursework or Assessment Specification

Module Details

Module Code	UFCF8S-30-2
Module Title	Advanced Software Development
Module Leader	
Module Tutors	
Year	2025 Jan
Component/Element Number	
Total number of assessments for this module	(1) Part-I: The first covers the analysis and design of a software system. (2) Part-II: The second covers the coding and testing of the design created in the first assignment.
Weighting	100%
Element Description	Group project: This assignment is to be completed in groups of up to three students.

Section 1: Overview of Assessment

This assignment assesses the following module learning outcomes (taken from the module specification):

- *Analyze problems in order to identify software-solution approaches and requirements for computer-based software-intensive systems.*
- *Compare and contrast software development methodologies and choose one suitable for a given application.*
- *Design, implement, test and manage reasonably sized concurrent and distributed software systems, working in a group, considering database and GUI components.*
- *Develop the necessary transferable skills – e.g. communication, delegation, openness, group decision making, flexibility, tolerance - to enable them to work in a group.*
- *Discuss the need for security in the context of system development.*
- *Evaluate emerging software development tools, technologies and methods, e.g. cloud-based development, DevOps and AI-based software development tools.*

The assignment is worth 100% of the overall mark for the module.

The assessment is designed to allow you to use your learning and play to your strengths. You can always seek advice and/or interim verbal feedback from your tutors during practical sessions.

Broadly speaking, the assignment requires you to design and implement a moderately realistic object-oriented system. You will produce detailed object models and designs from system requirements; use the modelling concepts provided by UML. You'll have a demo and Questions/Answer session with your tutor(s). You will then map the designs into code and perform unit testing using an automated testing tool. You will demonstrate your system and answer questions about the use of your approach and possible alternatives, technologies and tools.

The assignment is described in more detail in section 2.

This is a GROUP assignment and to be completed in groups of up to three students.

Working on this assignment will help you to understand more clearly the concepts, problems, and techniques of object-oriented systems, and how these can be used to design and implement a reasonably large size software system while working in a team (to a professional standard).

Section 2: Task Specifications

Note: All the tasks are based on the following scenario. Please read carefully all information contained within the following passage of text.

Scenario Overview

Hotel Management System (HMS) – Concept

Objective

The system will allow users to search for rooms, check availability, and make bookings. Hotel staff will manage room inventory, guest reservations, check-in/check-out processes, and billing.

Hotel Structure & Operations

Hotel Properties:

- The system manages multiple hotel locations (e.g., London, Paris, New York).
- Each hotel has various room types (e.g., Standard, Deluxe, Suite, Family Room).
- The system should allow new hotel properties.

Rooms & Booking Rules:

- Each room has a unique room number, type, capacity, price per night, and status (e.g., Available, Occupied, Cleaning, Out of Service).
- Guests can book a room for a specified duration (check-in and check-out dates).
- Cancellation fees apply if a booking is cancelled close to the check-in date.
- Guests can request additional services (e.g., airport transfer, spa access).
- A guest can book multiple rooms in a single reservation.

User Roles & Permissions:

1. Guests:

- Search for rooms, check availability and prices.
- Make, view, and cancel bookings.
- View booking history and invoices.
- Manage their user profile.

2. Front Desk Staff:

- Manage guest bookings (create, update, cancel).
- Perform guest check-in and check-out.
- Manage room status (e.g., assign room for cleaning).
- Process payments and generate bills.

3. Hotel Managers:

- Generate reports on occupancy, revenue, and guest demographics.
- Manage room rates and availability.
- Manage front desk staff accounts.

4. Admin:

- Maintain all user accounts (including managers).
- Configure system settings (e.g., add new hotel locations).

System Features & GUI Components

1. Room Search & Booking System

- Search rooms by date, location, room type, and capacity.
- Display room details, photos, and prices.
- Booking form to capture guest details and payment information.

2. Check-in & Check-out System

- Guest check-in form (confirm identity, process pre-authorization).
- Guest check-out processing (calculate final bill, include extra services, process final payment).

3. Staff Panel

- Dashboard showing current occupancy, arrivals, and departures.
- Manage room assignments and status.
- Process guest requests and service additions.

4. Guest Portal

- View current and upcoming bookings.
- Modify booking details (if allowed by policy).
- View past stays and download invoices.
- Update personal profile and payment methods.

5. Reports & Analytics

- Daily/Monthly/Yearly occupancy rates.
- Revenue reports by room type or service.
- Guest booking patterns.

Hotel Management System - Fee Structure

1. Room Rates (Per Night)

Room Type	Capacity	Price (GBP) - Off-Peak	Price (GBP) - Peak Season
Standard Double	2	120	180
Deluxe King	2	180	250
Family Suite	4	240	320
Penthouse	4	500	750

2. Cancellation Fees

Cancellation Notice	Cancellation Fee
More than 14 days before check-in	Free
3-14 days before check-in	50% of the first night's stay
Less than 72 hours before check-in	100% of the first night's stay
No-Show	100% of the entire booking value

3. Ancillary Service Fees (Optional)

Service	Fee
Airport Transfer (One-way)	50
Full English Breakfast (Per Person, Per Day)	20
Spa Access (Per Person, Per Day)	35
Late Check-out (until 2 PM)	40

Secure Development Requirements for Hotel Management System (HMS)

1. Authentication & Access Control

- Role-Based Access Control (RBAC):
 - Users should have access only to the features relevant to their role (Guests, Staff, Managers, Admins).
 - Implement least privilege principles to restrict unauthorized actions.
- Secure Login & Session Management:
 - Use secure cookies with HttpOnly and Secure flags.
- Account Lockout & Password Policy:
 - Enforce strong passwords (min 8 characters, upper/lowercase, number, special character).
 - Lock account after 5 failed login attempts (temporary lockout or CAPTCHA).
 - Force passwords change every 6 months for Admin and Manager accounts.
- Automatic Session Timeout:
 - Auto-logout inactive users after 15 minutes of inactivity.

2. Data Security & Encryption

- Database Security:
 - Implement database access controls to prevent unauthorized changes.

- Encrypt sensitive guest data at rest (e.g., payment card information placeholders, personal details).
- Secure Data Transmission:
 - Enforce HSTS (HTTP Strict Transport Security) to prevent downgrade attacks.
- Audit Logs & Monitoring:
 - Maintain audit logs for key activities (logins, bookings made, payments processed, check-in/out).
 - Log failed login attempts and suspicious activities for security monitoring.

Part-I (Weightage 40%)

- **Task 1.** Produce a case diagram to capture the functionality for the system to be built.
- **Task 2.** Produce a class diagram to meet all the requirements captured in the use case diagram.
- **Task 3.** Produce at least three sequence diagrams for some use cases that add information.

Part-II (Weightage 60%)

- **Task 4.** Write a joint report (300-700 words) outlining the steps that you followed to develop the system in relation to an Agile development episode and justify your choices. You should consider the benefits and challenges faced and may consider the following steps:

- **Strategy planning:** This is the initial phase for our system development process where we as developers need to identify system users to avoid building wrong solutions and identify the role and responsibilities of each team member

- **Continuous team iterations:** The development phases of our project are more flexible as we continuously iterate between planning and implementation compared to conventional development methodology which is too rigid and strict

Task 5. Develop the system using a suitable Object-Oriented Programming language (e.g., Python as we'll use Python programming language for this module) and Database(s) considering all the functionalities described above.

- You should design and implement suitable database(s) and fill it with suitable mock data for testing and demonstration.

- You should be creative and come up with your own User Interface Design for the different abovementioned GUI needs.
- You should also implement non-functional requirements (e.g., security measures) to a good standard.

Section 3: Deliverables

Electronically submitted deliverables include:

For Part I: Each student needs to upload the following delivery.

1. One PDF document that covers Task 1 to Task 3.

For Part II: Each student needs to upload the following deliverables in one .zip file. The naming standard of the zip file is WP1234567.zip where 1234567 is the student Id.

1. For task 4, a reflective group report in PDF format.
2. For task 5, the following deliverables should be compressed in a .zip file.
 - a. A software system based on the specifications in section 2, with source code in ZIP file (using e.g., 7Zip). This should contain all the files and folders for the full working system. All program files must have student ID and student name who has written the code. You should also include a list of additional Python packages (e.g., using pip list command) needed to run your software. Please do not include python distribution folder in your ZIP file. Provide a text file with instructions on how to start and use your software system e.g., any passwords etc.
 - b. relevant Database(s) (e.g. MySQL or MongoDB) dump.
3. For task 6, evidence of testing e.g., test cases, unit tests and outputs. These can be included in a separate PDF document.
4. For task 7, each student will write an individual report in PDF format.

Instructions for submission

You must submit your work *before* the stated deadline by electronic submission through Blackboard.

- Multiple submissions can be made to the portal, but only the final one will be accepted.
- **It is your responsibility to submit deliverables in the format stipulated above.** Your marks may be affected if your tutor cannot open or properly view your submission.

- **Do not leave submission to the very last minute.** Always allow time in case of technical issues. Also, save your work frequently to avoid losing your work at the last minute.
- The date and time of your submission is taken from the Blackboard server and is recorded when your submission is complete, not when you click Submit.
- **It is essential that you check that you have submitted the correct file(s),** and that each complete file was received. Submission receipts are accessed from the Coursework tab.
- UWE academic regulations for late submissions will be applied. Please check the module handbook.

Note: All submitted work will be checked for Plagiarism using SafeAssign or other suitable software/approaches. Every student undertaking an assignment or other piece of assessed work is required to take and will be deemed to have taken full responsibility for all the work submitted by the student. This includes responsibility for any assessment offence (e.g., Plagiarism, collusion) committed. Assessment of offences such as plagiarism or collusion will result in 0 marks.

Section 4: Marks Distribution and Marking Criteria

Part I weighs 40% and Part II weightage is 60%.

In common with all UWE standard undergraduate assignments, the pass mark for this assignment is 40%. The detailed marking criteria for this assignment is detailed in Appendix-A.

Section 5: Feedback mechanisms

On the spot verbal feedback during the demo session + as appropriate written feedback uploaded to Blackboard. Formative feedback provided in Part-I will be useful to complete Part-II of the assignment.

Please use Blackboard discussion forum to ask questions about this project.

Appendix A. Marking Criteria

Part-I (Weightage 40%)								
	0.0-2.9	3.0-3.9	4.0-4.9	5.0-5.9	6.0-6.9	7.0-8.4	8.5-10.0	Mark & Advice for Improvement
Use case diagram (10%)	Little or no diagram provided or diagram is mostly wrong (overall less than 30% complete).	Partial diagram provided but actors and/or use-cases/relationships are only partially correct (overall less than 40% complete).	Partial diagram provided, actors and/or use-cases/relationships are correct (overall less than 50% complete).	Almost complete diagram provided, actors and/or use-cases/relationships are only partially correct (overall less than 60% complete).	Almost complete diagram provided, actors and/or use-cases/relationships are correct. (overall, less than 70% complete)	Complete diagram provided, actors and/or use-cases/relationships are partially correct (overall less than 85% complete)	Complete diagram provided, actors and/or use-cases/relationships are correct (100% complete)	
Class diagram (10%)	Little or no diagram provided or diagram is mostly wrong (overall less than 30% complete).	Partial diagram provided but Class naming convention, Class relationships, Attributes/methods of the classes and their visibility and type are only partially correct (overall, less than 40% complete).	Partial diagram provided but Class naming convention, Class relationships, Attributes/methods of the classes and their visibility and type are correct (overall, less than 50% complete).	Almost complete diagram provided, Class naming convention, Class relationships, Attributes/methods of the classes and their visibility and type are only partially correct (overall less than 60% complete).	Almost complete diagram provided, Class naming convention, Class relationships, Attributes/methods of the classes and their visibility and type are correct (overall less than 70% complete).	Complete diagram provided, Class naming convention, Class relationships, Attributes/methods of the classes and their visibility and type are partially correct	Complete diagram provided, Class naming convention, Class relationships, Attributes/methods of the classes and their visibility and type are correct (100% complete)	

[illegible]

	Part-II (Weightage 60%)							
	0.0-2.9	3.0-3.9	5.0-5.9	5.0-5.9	6.0-6.9	7.0-8.4	8.5-10.0	Mark & Advice for Improvement
Agile development (10%)	Little or no description provided (overall less than 30% complete).	Partial description provided but texts/justification are not written reasonably accurately (overall less than 40% complete).	Partial description provided, texts/justification are written reasonably accurately (overall less than 50% complete).	Almost complete description provided but texts/justification are not written reasonably accurately (overall less than 60% complete).	Almost complete description provided, texts/justification are written roughly accurately (overall less than 70% complete).	Almost complete description provided, texts/justification are written roughly accurately (overall less than 70% complete).	Complete description provided, texts/justification are written very well and accurately (100% complete).	
	0.0-6.0	6.1-8.0	8.1-10.0	10.1-12.0	12.1-14.0	14.1-17.0	17.1-20.0	Mark & Advice for Improvement
Implementation (20%)	Little or no development (overall, less than 30% complete).	Partial development compiles and runs, but GUI displays input and output messages only partially correctly (overall less than 40% complete).	Partial development compiles and runs, GUI displays input and output messages only partially correctly (overall less than 50% complete).	Partial development, compiles and runs, but GUI displays input and output messages only partially correctly (overall less than 60% complete).	Almost complete development, compiles and runs, GUI displays input and output messages only partially correctly (overall less than 70% complete).	Complete development including security measures, compiles and runs, but GUI displays input and output messages only partially correctly. (overall less than 85% complete).	Complete development including security measures, compiles and runs, GUI displays input and output messages correctly. One or more Databases are used for persistent storage. (100% complete).	

[illegible]